Homography

1. Homography

```
# -*- coding: utf-8 -*-
"""
Created on Sat Feb 21 06:59:20 2026

@author: reina
"""

import numpy as np
import cv2
import time
import matplotlib.pyplot as plt


def test_homography(H, pts_src):
    pts_src_ = np.hstack((pts_src, np.ones((pts_src.shape[0], 1))))
    pts_dst_ = (H @ pts_src_.T).T
    return pts_dst_[:, :2] / np.repeat(pts_dst_[:, -1][:, np.newaxis], 2, 1)


# TODO 1 (15%): Implement calc_homography()
def calc_homography(pts_src, pts_dst):
  A = np.zeros((8, 9), dtype=np.float64)  # for 4 known points

  for i in range(4):
      x_src = pts_src[i, 0]; y_src = pts_src[i, 1]
      x_dst = pts_dst[i, 0]; y_dst = pts_dst[i, 1]
      A[2*i, 0:3] = np.array([-x_src, -y_src, -1])
      A[2*i, 6:9] = np.array([x_src*x_dst, y_src*x_dst, x_dst])
      A[2*i+1, 3:6] = np.array([-x_src, -y_src, -1])
      A[2*i+1, 6:9] = np.array([x_src*y_dst, y_src*y_dst, y_dst])
  u, s, vh = np.linalg.svd(A)
  H = vh[-1, :].T
  H = H.reshape((3, 3))
  return H


def project_img(H, img_src, img_dst):
    ht_src, wid_src, _ = img_src.shape
    ht_dst, wid_dst, _ = img_dst.shape
    for j in range(ht_src):
        for i in range(wid_src):
            pt = np.array([[i, j, 1]])  # shape (1, 3)
            proj_pt = (H @ pt.T).T
            proj_pt_ = proj_pt[:, :2] / proj_pt[:, -1]
            x = int(proj_pt_[0, 0]); y = int(proj_pt_[0, 1])
            if y > 0 and y < ht_dst and x > 0 and x < wid_dst:
                img_dst[y, x, :] = img_src[j, i, :]

    # return img_dst


def inverse_project_img(G, img_src, img_dst, pts_dst):
    ht_src, wid_src, _ = img_src.shape
```

```
    ht_dst, wid_dst, _ = img_dst.shape
    poly = cv2.fillPoly(img_dst, [pts_dst], (0, 0, 0))
    # print(poly.shape)
    # cv2.namedWindow('poly', cv2.WINDOW_NORMAL)
    # ht_window, wid_window = int(ht_dst*0.3), int(wid_dst*0.3)
    # cv2.resizeWindow('poly', ht_window, wid_window)
    # cv2.imshow('poly', poly)
    # cv2.waitKey(0)
    # cv2.destroyAllWindows()
    for j in range(ht_dst):
        for i in range(wid_dst):
            if (poly[j, i, :] == (0, 0, 0)).all():
                pt = np.array([[i, j, 1]])  # shape (1, 3)
                proj_pt = (G @ pt.T).T
                proj_pt_dehom = proj_pt[:, :2] / proj_pt[:, -1]
                x = int(proj_pt_dehom[0, 0]); y = int(proj_pt_dehom[0, 1])
                if y > 0 and y < ht_src and x > 0 and x < wid_src:
                    img_dst[j, i, :] = img_src[y, x, :]


if __name__ == '__main__':
    img_src = cv2.imread("monet.jpg")
    img_dst = cv2.imread("painting.jpg")
    ht_src, wid_src, _ = img_src.shape
    ht_dst, wid_dst, _ = img_dst.shape
    pts_src = np.array([    [0,0],[wid_src, 0],[wid_src,ht_src], [0,ht_src]])
    pts_dst = np.array([[299,515],  [1776,360],     [1680,1834], [239,1615]])
    print('source_shape:', ht_src, wid_src)
    print('destination_shape:', ht_dst, wid_dst)

    cv2.namedWindow('monet', cv2.WINDOW_NORMAL)
    ht_window = int(ht_src*0.3); wid_window = int(wid_src*0.3)
    cv2.resizeWindow('monet', wid_window, ht_window)
    cv2.imshow('monet', img_src)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

    cv2.namedWindow('painting', cv2.WINDOW_NORMAL)
    ht_window = int(ht_dst*0.3); wid_window = int(wid_dst*0.3)
    cv2.resizeWindow('painting', wid_window, ht_window)
    cv2.imshow('painting', img_dst)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

    H = calc_homography(pts_src, pts_dst)
    H = H/H[2,2]
    print(H)
    print(test_homography(H, pts_src))

    project_img(H, img_src, img_dst)

    cv2.namedWindow('painting', cv2.WINDOW_NORMAL)
    ht_window = int(ht_dst*0.3); wid_window = int(wid_dst*0.3)
    cv2.resizeWindow('painting', wid_window, ht_window)
    cv2.imshow('painting', img_dst)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

    G = calc_homography(pts_dst, pts_src)
```

```
    G = G/G[2,2]

    inverse_project_img(G, img_src, img_dst, pts_dst)

    cv2.namedWindow('painting', cv2.WINDOW_NORMAL)
    ht_window = int(ht_dst*0.3); wid_window = int(wid_dst*0.3)
    cv2.resizeWindow('painting', wid_window, ht_window)
    cv2.imshow('painting', img_dst)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

2. Image stitching

```
# -*- coding: utf-8 -*-
"""
Created on Sat Feb 21 12:42:47 2026

@author: reina
"""

import cv2
import numpy as np

def calc_homography(pts_src, pts_dst):
  A = np.zeros((8, 9), dtype=np.float64)  # for 4 known point
                                          # correspondences (2D-2D)

  for i in range(4):
      x_src = pts_src[i, 0]; y_src = pts_src[i, 1]
      x_dst = pts_dst[i, 0]; y_dst = pts_dst[i, 1]
      A[2*i, 0:3] = np.array([-x_src, -y_src, -1])
      A[2*i, 6:9] = np.array([x_src*x_dst, y_src*x_dst, x_dst])
      A[2*i+1, 3:6] = np.array([-x_src, -y_src, -1])
      A[2*i+1, 6:9] = np.array([x_src*y_dst, y_src*y_dst, y_dst])
  u, s, vh = np.linalg.svd(A)
  H = vh[-1, :].T
  H = H.reshape((3, 3))
  return H


if __name__ == '__main__':
    img_dst = cv2.imread('library1.jpg')
    img_src = cv2.imread('library2.jpg')
    img_dst_copy = img_dst.copy()
    img_src_copy = img_src.copy()
    ht_dst, wid_dst, _ = img_dst.shape
    ht_src, wid_src, _ = img_src.shape
    pts_dst = np.array([[300, 68], [296, 217], [454, 40], [449, 217]])
    pts_src = np.array([[69, 55], [64, 211], [226, 35], [219, 209]])
    # View key points
    for i in range(4):
        cv2.circle(img_dst_copy, (pts_dst[i, 0], pts_dst[i, 1]), 2, (0, 0, 255), 5)
        cv2.circle(img_src_copy, (pts_src[i, 0], pts_src[i, 1]), 2, (0, 0, 255), 5)

    cv2.imshow('library1', img_dst_copy)
    cv2.imshow('library2', img_src_copy)
    cv2.waitKey(0)
```

```
cv2.destroyAllWindows()

# # Compute homography for direct projection
# H = calc_homography(pts_src, pts_dst)
# img_canvas = np.zeros((ht_dst, wid_dst * 2, 3), dtype=np.uint8)
# h, w, c = img_canvas.shape
# for j in range(ht_src):
#     for i in range(wid_src):
#         pt = np.array([[i, j, 1]])
#         proj_pt = (H @ pt.T).T
#         proj_pt_dehom = proj_pt[:, :2] / proj_pt[:, -1]
#         x = int(proj_pt_dehom[0, 0]); y = int(proj_pt_dehom[0, 1])
#         if y > 0 and y < h and x > 0 and x < w:
#             img_canvas[y, x, :] = img_src[j, i, :]

# Compute homography for inverse projection
G = calc_homography(pts_dst, pts_src)
img_canvas = np.zeros((ht_dst, wid_dst * 2, 3), np.uint8)
h, w, c = img_canvas.shape
for j in range(ht_dst):
    for i in range(wid_dst):
        pt = np.array([[i + wid_dst, j, 1]])
        proj_pt = (G @ pt.T).T
        proj_pt_dehom = proj_pt[:, :2] / proj_pt[:, -1]
        x = int(proj_pt_dehom[0, 0]); y = int(proj_pt_dehom[0, 1])
        if y > 0 and y < ht_src and x > 0 and x < wid_src:
            img_canvas[j, i + wid_dst, :] = img_src[y, x, :]

# print(np.max(img_warped))
cv2.imshow('img_warped', img_canvas)
cv2.waitKey(0)
cv2.destroyAllWindows()

img_canvas[:, :wid_src, :] = img_dst
cv2.imshow('img_stitched', img_canvas)
cv2.waitKey(0)
cv2.destroyAllWindows()
```